

Module : Kotlin (M202)

TP N° 19

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVOAM201 - DEVOAM202 Niveau : 2ème année

Exercice 1 : Étapes pour configurer un projet avec des flavors

1. Configuration des flavors dans Gradle

Dans le fichier `build.gradle` du module (généralement `app/build.gradle`), tu vas ajouter la configuration des flavors.

```
android {  
  
    compileSdkVersion 33  
  
    defaultConfig {  
  
        applicationId "com.example.flavorexample"  
  
        minSdkVersion 21  
  
        targetSdkVersion 33  
  
        versionCode 1  
  
        versionName "1.0"  
  
    }  
  
    buildTypes {  
  
        release {
```

```
minifyEnabled false

proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'

}

}

flavorDimensions "flavor"

productFlavors {

    chocolate {

        dimension "flavor"

        applicationIdSuffix ".chocolate"

        versionNameSuffix "-chocolate"

    }

    strawberry {

        dimension "flavor"

        applicationIdSuffix ".strawberry"

        versionNameSuffix "-strawberry"

    }

    vanilla {

        dimension "flavor"
```

```
        applicationIdSuffix ".vanilla"  
  
        versionNameSuffix "-vanilla"  
  
    }  
  
}  
  
}
```

- **flavorDimensions** : Un projet peut avoir plusieurs dimensions (par exemple, "flavor" et "API"), mais ici, nous utilisons une seule dimension, nommée "flavor".
- **productFlavors** : Nous définissons trois flavors : **chocolate**, **strawberry**, et **vanilla**.
 - **applicationIdSuffix** ajoute un suffixe à l'ID de l'application pour chaque flavor (utile pour installer plusieurs flavors sur le même appareil).
 - **versionNameSuffix** ajoute un suffixe à la version de l'application.

2. Personnaliser les ressources pour chaque flavor

Tu peux aussi avoir des ressources (comme des images, des chaînes de caractères, etc.) spécifiques à chaque flavor. Crée des dossiers séparés pour chaque flavor dans **src**.

Dans le répertoire **app/src**, crée trois nouveaux dossiers pour chaque flavor : **chocolate**, **strawberry**, et **vanilla**. La structure du projet ressemblera à ceci :

```
app/  
├── src/  
│   ├── main/  
│   └── chocolate/
```



3. Personnaliser les chaînes de caractères (strings.xml)

Dans chaque dossier flavor, tu peux avoir des ressources spécifiques. Par exemple, dans `chocolate/res/values/strings.xml`, tu peux définir une chaîne spécifique pour le flavor Chocolate.

- **`app/src/chocolate/res/values/strings.xml` :**

```

<resources>

    <string name="app_name">Chocolate Flavor</string>

</resources>

```

- **`app/src/strawberry/res/values/strings.xml` :**

```

<resources>

    <string name="app_name">Strawberry Flavor</string>

```

</resources>

- **app/src/vanilla/res/values/strings.xml :**

<resources>

<string name="app_name">Vanilla Flavor</string>

</resources>

Ces chaînes seront utilisées pour nommer l'application selon le flavor sélectionné lors de la compilation.

4. Utiliser les flavors dans ton code (MainActivity.kt)

Dans ton activité principale (par exemple **MainActivity.kt**), tu peux utiliser les ressources spécifiques au flavor sélectionné.

```
import android.os.Bundle
```

```
import androidx.appcompat.app.AppCompatActivity
```

```
import android.widget.TextView
```

```
class MainActivity : AppCompatActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main)
```

```
        val textView: TextView = findViewById(R.id.textView)
```

```
        textView.text = getString(R.string.app_name) // Affiche le nom selon le flavor
```

}

}

5. Build et exécuter les flavors

Tu peux maintenant construire et exécuter chaque flavor directement depuis Android Studio.

- Dans le menu déroulant de l'interface Android Studio (en haut à droite, à côté du bouton "Run"), tu trouveras toutes les variantes de build, comme :
 - chocolateDebug
 - strawberryDebug
 - vanillaDebug
 - chocolateRelease
 - etc.

Sélectionne le flavor souhaité et exécute-le.

6. Ajout de dépendances spécifiques pour chaque flavor (facultatif)

Tu peux aussi ajouter des dépendances spécifiques pour chaque flavor. Par exemple, si tu veux que le flavor **vanilla** utilise une dépendance particulière, tu peux le faire dans **build.gradle** :

```
productFlavors {  
  
    chocolate {  
  
        dimension "flavor"  
  
        applicationIdSuffix ".chocolate"  
  
        versionNameSuffix "-chocolate"  
  
    }  
}
```

}

strawberry {

dimension "flavor"

applicationIdSuffix ".strawberry"

versionNameSuffix "-strawberry"

}

vanilla {

dimension "flavor"

applicationIdSuffix ".vanilla"

versionNameSuffix "-vanilla"

}

}

dependencies {

vanillaImplementation 'com.example.library:vanilla-lib:1.0'

strawberryImplementation 'com.example.library:strawberry-lib:1.0'

chocolateImplementation 'com.example.library:chocolate-lib:1.0'

}